

AgentWorkShop: Node-Native Binding and Governed Write Paths for Agent Teams in Industrial Supervision

Anonymous Authors

Abstract—Bringing language-model agents into industrial supervisory control requires associating the semantics, authority, and history of each command across heterogeneous devices, multiple agents, and changing operating recipes. We present AGENTWORKSHOP, a node-native framework in which explicit member-to-node bindings determine each agent’s tools, source-dependent checks govern agent-issued writes, and attributable records with an independent backstop keep authorization, device execution, and process judgment distinguishable. The implementation covers five protocol drivers and registers fourteen agent engines as an inventory rather than evaluated controllers. Evidence comes from one archived run over three simulated production scenarios and real software protocol transports, including a 49-signal biaxial-film line. The run passes 75 of 75 checks with no warnings or failures, three of them carrying write sub-checks skipped because their nodes expose no writable setpoint. Its four writable protocol fixtures reject 24 of 24 predefined out-of-constraint probes and block none of four designated legal writes; a second scenario rejects 9 of 9 out-of-constraint probes without false blocks. A fixed non-LLM controller over three seeds reaches 0.963–0.972 of an offline grid-search reference, depending on whether the end score or the final iterate is used. A scripted team mission, with the LLM path disabled, reduces biaxial-film thickness from 28.10 to 25.68 μm , inside a $25.0 \pm 0.7 \mu\text{m}$ objective, using three governed writes on two protocols. These are suite-coverage and single-run capability results on simulated plants; they establish neither physical safety nor agent performance.

Index Terms—Industrial informatics, multi-agent systems, supervisory control, human-in-the-loop, heterogeneous protocols, large language models.

I. INTRODUCTION

LARGE language models (LLMs) are being explored for industrial diagnosis, engineering assistance, and control-program generation [1], [2], [3], [4]. Recent systems also connect agents to simulated controllers or real machines [5], [6]. For a production-line deployment, the engineering problem is therefore not whether an LLM can issue a command but how to preserve its meaning, authority, and history across heterogeneous equipment, multiple agents, and changing operating recipes.

Consider a team asked to correct film thickness within pressure and temperature limits: one member may inspect telemetry, another propose a setpoint, and a lead coordinate the task. A protocol connection alone does not identify which member may change which node, which recipe applies to the current batch, whether a human must approve, or whether the subsequent observation measures the commanded register or the process response. Implementing these associations separately for each application duplicates configuration and makes

failures hard to reconstruct; conversely, one shared interface does not by itself establish atomic execution, physical safety, or valid optimization. Fig. 1 contrasts this application-specific pattern with the binding and production context used here.

We present AGENTWORKSHOP, a configurable framework for this supervisory integration problem: an industrial node combines its acquisition or setpoint interface with engineering semantics, and an explicit binding grants a deployed channel member access to that node; a line/product/recipe/run model supplies the production context shared by observations and actions. The platform core hosts agent orchestration and industrial supervision together, while engine subprocesses, optional external stores, and field endpoints retain their own boundaries. This organization reduces application-specific routing within the supported driver set without equating shared state with an end-to-end transaction.

The agent loop operates at supervisory, seconds-to-minutes timescales; regulatory control and plant-side protective functions remain in PLCs. Manual bindings require an approval decision; automatic bindings omit that wait but retain the applicable execution checks; write attribution and separately triggered compensation make a failed proposal, a failed write, and an unsuccessful adjustment inspectable as different events.

The contributions are threefold:

- 1) A **node-native integration model** connecting heterogeneous acquisition and control nodes, deployed agent teams, and versioned production runs through explicit capabilities and shared identifiers (Sec. III).
- 2) An **implemented supervisory workflow** that separates team coordination, source-dependent write checks, agent-submitted judgments, physical compensation, and human decisions; its seven mechanisms and the cross-cutting HITL boundary are described with their actual scopes and exceptions (Sec. IV).
- 3) A **traceable, layered evaluation** organized around the latest machine-readable integrated archive. The run passes 75/75 checks with no warnings or failures; its four writable protocol fixtures reject 24/24 predefined out-of-constraint probes with 0/4 false blocks, and a second film-line scenario rejects 9/9 out-of-constraint probes with 0/3 false blocks. Three fixed-controller seeds reach $J/J^* = 0.963\text{--}0.972$ across the two conventions the archive uses, and a scripted team mission with the LLM path disabled reduces biaxial-film thickness from 28.10 to 25.68 μm with three governed writes on two protocols (Sec. V).

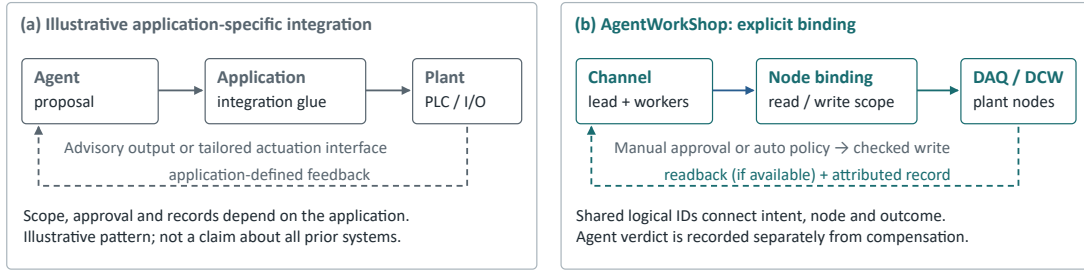


Fig. 1. Integration problem and proposed organization. The left panel illustrates advisory or application-specific integration, not a universal limitation of prior systems. The right panel associates agent-team membership, explicit node capabilities, production context, and execution feedback. Arrows distinguish information exchange from setpoint requests; PLC regulation remains outside the agent loop.

These contributions correspond to the evaluation’s evidence layers (Sec. V, Table I): the integration model by the multi-stack and lifecycle suites, the mechanisms by the governance probes, and the assembled workflow by the fixed-controller and mission suites. Each layer is reported separately rather than pooled into one score. The contribution is a systems contribution: the paper specifies an integration model, implements it end to end, and reports what that implementation does and does not guarantee. It proposes no new control law, learning method, or safety certification argument, and claims reusable supervisory integration within the tested settings, not certified safety, general industrial autonomy, or LLM-controller superiority.

II. RELATED WORK

A. Industrial Agents and Closed-Loop Applications

Agent-based distributed manufacturing control has a two-decade history [7]; what is contributed here is the binding of LLM-team membership to node capability. Industrial LLM applications span industrial knowledge-graph construction [8] and diagnosis [9], PLC program generation and verification [3], [4], and digital-twin configuration [10]. Not all industrial agents are advisory: natural-language interfaces have controlled machines over OPC UA [6], LLM agents have been studied in simulated fault-tolerant control [5], and InstructMPC uses language-derived context for disturbance prediction in model-predictive control [11]. Closest to our setting, agent-based supervision has been studied for service-oriented industrial cyber-physical systems [12], and LLM-based multi-agent systems coordinate shopfloor manufacturing tasks [13]; neither binds individual team members to specific nodes with source-dependent execution checks and line/product/recipe/run attribution.

B. Orchestration, Industrial Models, and Execution Governance

AutoGen, LangGraph, and CrewAI organize agent tasks and tool use [14], [15], [16], and MCP and A2A standardize tool and inter-agent interfaces [17], [18]; industrial integration additionally requires node semantics, driver behavior, and operating context. We use a line/product/recipe/run organization informed by ISA-95 and ISA-88 [19], [20]: line and product correspond to level-3 production work units and product

definitions, run to production-performance records, and recipe to the ISA-88 master-recipe idea with grade and operating-point semantics on continuous lines; we do not implement the complete procedural models of either standard. The device-twin terminology presumes manufacturing digital twins whose assets carry structured semantic descriptions [21], and the node semantic card is analogous to an Asset Administration Shell submodel [22]. A goal-driven unit’s observe–hypothesize–write–re-observe cycle likewise resembles autonomous chemical research with LLMs [23]; our writes are governed industrial setpoints rather than self-directed experiment planning.

The ingredients of a governed write—node-scoped permission, range validation, approval, and an audit trail—have counterparts in mature DCS/SCADA/MES practice, typically enforced as role- and application-level controls under IEC 62443-3-3 [24]. LLM-mediated writes additionally require member-instance-level binding, semantic linkage between a write and the verdict or optimization record that produced it, and attribution across engine boundaries. Hard action shielding and control-barrier methods enforce formally specified admissible actions [25]; the rollback manager and rule-based backstop are engineering instances of run-time assurance and simplex-style architectures [26], and are narrower implementation-level checks rather than substitutes for certified plant protection. Human supervision also requires a clear division of responsibility, not merely an approval dialog [27], [28].

C. Evaluation of Tools and Industrial Control

AgentBench and consequential tool-use benchmarks evaluate agent behavior and policy compliance [29], [30]. Industrial datasets such as TEP and SWaT support process-monitoring research [31], [32], but replay is not evidence of successful physical actuation, so we separate source checks, mock-driver probes, software-protocol integration, fixed-controller optimization, and exploratory LLM runs. The threat model of indirect prompt injection [33]—a compromised agent writing through legitimate tools—is what the write-side checks are designed to contain, although our evaluation includes no adversarial LLM probes.

III. SYSTEM ARCHITECTURE

AGENTWORKSHOP integrates multi-agent reasoning with industrial supervision around a common representation of

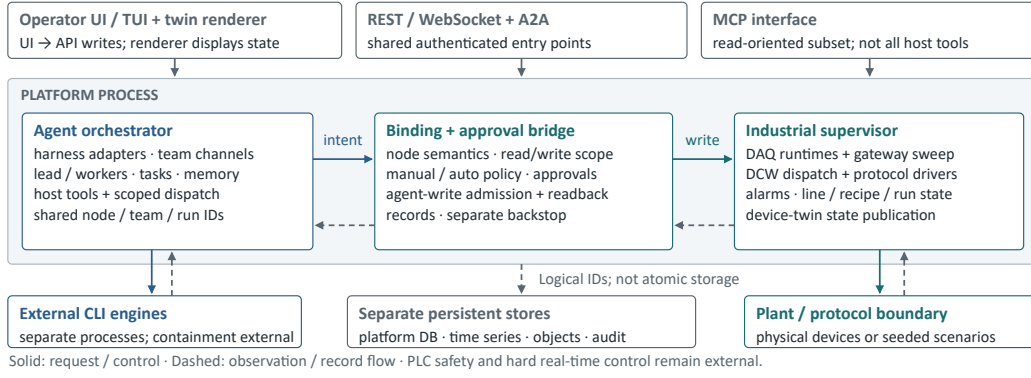


Fig. 2. System architecture and execution boundaries. The co-resident platform core connects agent channels, semantic nodes, industrial controllers, and operator services. CLI/OMP engines and storage services extend beyond that process. Shared identifiers support attribution across the logical data model; they do not make storage or physical actuation transactional.

production assets and operating context. Its organizing object is the node: an acquisition or control endpoint with engineering semantics, device connections, line membership, and agent bindings. A channel organizes the agents that interpret these endpoints, controllers mediate plant interactions, and Fig. 2 locates these components and execution boundaries.

A. A Shared Supervisory Core

The Node.js core hosts channel routing, task scheduling, industrial services, protocol adapters, and platform interfaces: REST and WebSocket endpoints serve operator applications, MCP and A2A interfaces support external integration, and a plugin host extends platform capabilities. These surfaces reuse platform objects, but their exposed operations and permission checks differ, and an agent's industrial authority derives from its node bindings, not from channel access or protocol-endpoint availability. CLI engines and the platform's bundled OMP agent harness execute in spawned processes, TimescaleDB and MinIO are external, and connection establishment, queued protocol operations, and readback introduce asynchronous boundaries. Fast regulation and protection therefore remain with the plant controller and its safety equipment.

B. Reasoning, Plant Interaction, and Operational Context

On the reasoning side, a channel combines a lead, worker instances, a task tree, messages, and retrievable memory. Engine adapters allow different reasoning implementations to participate without changing a bound node's identity or the controller interface.

On the plant side, acquisition nodes turn protocol readings into engineering quantities, control nodes translate engineering setpoints into device commands and readback results, per-node runtimes manage timing and in-flight operations, and controller services supply gateway state, recipe context, and write bookkeeping. Nodes and devices have many-to-many configuration associations, but association is distinct from execution routing: a control-node write invokes its configured driver and connection settings rather than iterating over its associated devices, so the interface follows process meaning without implying multi-device actuation by one write.

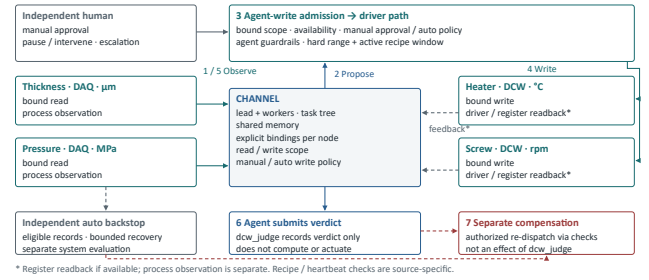


Fig. 3. The mechanisms in their organizational setting and closed loop. Numbered items are loop steps, not a one-to-one list of the subsections below; steps 1 and 5 share the observation arrows. The channel coordinates agents that observe bound nodes, propose setpoints, and pass admission checks before write and readback. Step 6 is the agent's own verdict, the independent backstop is a second and separate judgment path, and step 7 is compensation that neither path performs. `dcw_judge` records a verdict only: it neither computes the conclusion from telemetry nor executes any action.

C. Logical Continuity Across a Production Run

Lines, products, recipes, and runs provide the context in which measurements and interventions become comparable. Their identifiers connect platform state, time series, image frames, operational events, and optimization records across heterogeneous stores: SQLite and file-backed repositories for platform state, TimescaleDB for measurements, MinIO for objects. These constitute one logical data model rather than one transaction domain.

Operators participate through approvals, line operations, and monitoring. Section IV details these mechanisms and marks where their guarantees end.

IV. CORE MECHANISMS

Seven mechanisms carry the integration model: node connection and creation (Section IV-A), the agent channel (Section IV-B), node control execution (Section IV-C), production-line operations (Section IV-D), agent closed-loop control (Section IV-E), recipe management and rollback (Section IV-F), and the process-parameter mapping layer (Section IV-G), plus the cross-cutting HITL boundary (Section IV-H). The evidence is not uniform: the out-of-constraint write probes exercise the hard-range and recipe-window checks directly, the lifecycle

Fig. 4. Operator-facing workflow using archived interface crops: node commissioning, device association, team deployment, and optimization-record inspection. Device association and agent capability binding are distinct operations, though the current interface does not enforce distinct operator roles for them. A displayed judgment records a disposition; it does not independently certify process improvement or physical recovery.

and mission checks exercise rollback and approval, and the ownership and cooldown checks are traversed by every governed agent write (Sections V-C, V-F, and V-E). The probes are fixed protocol cases, not model-generated adversarial inputs.

A. Node Connection and Creation

A node separates the meaning of an industrial variable from its transport address. Commissioning binds device integration information to an acquisition or control configuration, tests connectivity, and supplies units, calibration, limits, and production-line membership. Drivers support Modbus TCP, Modbus RTU over TCP, OPC UA [34], MQTT, HTTP/REST, plus a mock implementation for testing. Creation is not authorization to operate: operator APIs and agent-facing paths carry different permission conditions, and an industrial-tool binding refers to a concrete channel-member instance, not a reusable template, so deploying a template elsewhere does not transfer the binding. That last property is a design consequence rather than a measured result; the archive exercises binding denial only on the parameter surface.

The semantic card presents the node's physical quantity, unit, precision, range, current value, freshness, and available recipe context. Keeping a setpoint change within 2 % of span is prompt guidance, not a controller-enforced step limit: the biaxial mission's first write moves 3.6 % of span and nothing rejects it. Per-node acquisition runtimes manage due times and in-flight sampling, and a rotating gateway sweep apportions that work across intervals, supporting supervisory observation rather than hard real-time sampling.

B. The Agent Channel: Team, Supervision, and Memory

A channel contains a lead, worker instances, a task tree, and message history. Routing resolves recipients within the channel and loads runtimes as needed. The task tree distinguishes submitted from completed work, but those states describe collaboration progress, not device execution: a completed task is not permission to write and not evidence that a write occurred. The scheduler combines periodic supervision with event wake-up and idle backoff, summarizing tasks, queues, member activity, and recent messages, and deterministic rules handle dispatch, retry, and stalled work; scheduler progress cannot resolve uncertainty about a plant response, which

requires observations and an explicit judgment. Channel memory preserves experience and reusable knowledge for later retrieval, so a later worker recovers earlier rationale instead of inferring it from a current setpoint, but a retrieved memory is contextual evidence, not an authorization mechanism or an executable safety specification. Industrial tools resolve the invoking member's bindings at request time, on the judge and rollback paths as well as the write path; the parameter-layer check `agent-param-tools` in Section V-C exercises that denial, only for the parameter surface and with a scripted caller.

C. Node Control Execution

The execution path separates an authorized request from a verified device result. An ordinary agent setpoint request resolves a control-node binding and, in manual mode, waits for approval; the controller and node runtime then apply pause, ownership, cooldown, range, and write-exclusion checks before protocol execution.

Let $S_n = [l_n, u_n]$ be the node range, r the recipe identifier, $W_{r,n}$ the parameter window that recipe r prescribes for node n , q the optional recipe-run identifier supplied with a request, and P_r the nodes with a parameter in that recipe. The effective constraint at controller entry is

$$C(n, r, q) = \begin{cases} S_n \cap W_{r,n}, & r \neq \emptyset, q = \emptyset, n \in P_r, \\ S_n, & \text{otherwise.} \end{cases} \quad (1)$$

An unspecified window endpoint is unbounded. Equation (1) describes normal controller requests, not every low-level executor invocation: recipe application supplies q and so bypasses the active-window branch, hold heartbeats share write exclusion but call the executor directly without re-entering all controller checks, and ownership and cooldown checks depend on source metadata and are not applied universally.

Calibration maps engineering values to the device domain and readback into physical units. The runtime's tolerance is

$$\tau_n = \max(0.5 \cdot 10^{-d_n}, 0.005(u_n - l_n)), \quad |v_{rb} - v| \leq \tau_n, \quad (2)$$

with display precision d_n ; for linear calibration the executor converts it to the absolute scale before comparison. A successful readback supports agreement with the commanded value, not proof that the process reached its desired operating

Algorithm 1 Ordinary bound-agent setpoint request

Require: Member a , node n , engineering target v

- 1: $b \leftarrow \text{FINDBINDING}(a, n)$; reject if absent
- 2: **if** $b.\text{mode} = \text{manual}$ **then**
- 3: Await approval; reject on denial or timeout; recheck binding
- 4: **end if**
- 5: Reject if gateway paused or node disabled
- 6: Check agent ownership and directional cooldown
- 7: Reject if $v \notin C(n, r, \emptyset)$; validate range
- 8: Acquire node write exclusion; await connection, write, and readback
- 9: Release exclusion; preserve execution outcome
- 10: **if** execution reports success **then**
- 11: Attempt anchor, record, and operations bookkeeping
- 12: **end if**
- 13: **return** execution result and available record identifiers

condition, so communication failure and readback mismatch stay distinct from process-effect judgment. With no readback available, displaying the commanded value as a fallback does not establish measured agreement.

Algorithm 1 summarizes this ingress: asynchronous connection and protocol queuing separate the checks from the device action, so no co-residency or node write exclusion supplies check–act atomicity against a changing recipe state. A successful write triggers journal anchors and, for relevant sources, optimization records; bookkeeping errors are caught without reversing the result, so attribution is traceability, not an atomic commitment across storage and the device.

D. Production-Line Operations

Line operations associate products, recipes, and runs with the nodes that generate measurements or accept setpoints. The run gives a batch identity, so a temperature observed under one product’s recipe should not become evidence about another product’s operating window. Starting a line validates its recipe and product, creates a run, installs it as active, and writes its parameters individually; an active run is not a certificate that every parameter applied, and individual outcomes must be examined when initialization fails partway. Acquisition is gated by active batches, so opening a run establishes the context in which measurements are collected; stopping the line clears that context, closes up to 500 open optimization records through the current stop hook, and requests acquisition-offline status. Closure beyond that limit is not guaranteed, and the operation is a supervisory batch command, not a PLC emergency stop.

Monitoring combines live values, node conditions, alarms, and events with historical measurements and frames, so a response can be inspected over time. The device-twin scene renders these states without issuing protocol commands, although Town operator web controls can submit API writes. Neither the scene nor a stopped batch is a physical safety barrier; between batches the run gate limits acquisition while plant-side protection remains independent.

E. Agent Closed-Loop Control

The feedback sequence is observation, hypothesis, intervention, judgment, and retention or recovery. The agent inspects bound-node context and acquisition history, states an expected effect, and proposes an engineering setpoint. After a successful value-changing write, the rollback manager creates an anchor and an optimization record holding the node, actor, hypothesis, parameter change, and policy, with baseline statistics filled asynchronously from preceding observations. After a five-second same-target deduplication check, a subsequent successful write supersedes the node’s open record; an unchanged-value write creates no replacement anchor or record, while a value-changing agent or rollback write starts a new observation record. Node-scoped bookkeeping, not one controlled experiment per agent across the plant. Before an agent write, an open record belonging to another agent blocks entry unless superseded as stale, and a recent rollback prevents an agent from immediately moving in the same direction away from the restored value. These checks reduce conflicting interventions but do not exclude manual or recipe-origin writes. Defaults are a 600 s baseline lookback, a 300 s cooldown, and a 30 min stale-record interval.

The agent then invokes `dcw_judge` with a record identifier, verdict, and reason. The tool validates and records the supplied conclusion, so a *keep*, *rollback*, or *uncertain* judgment is attributable reasoning rather than an established process result. A *keep* judgment closes the record and can mark a good state, whereas *rollback* and *uncertain* judgments leave it open; restoring a setpoint requires a separate rollback operation, and uncertainty must not be read as successful optimization.

An independent backstop periodically evaluates eligible open records, waiting at least 120 s by default and rechecking every 30 s. Acquisition channels matching the control node’s primary-device alias are preferred, otherwise falling back to the line. Historical query results are compared with configured acquisition monitoring windows, and at least three accumulated breaches trigger a rollback judgment. The queried observations, the aggregation, and data availability all constrain this evidence, and the intervals alone prove no worst-case physical recovery deadline. Under an automatic-rollback policy, the backstop executes recovery only while the node’s count of records marked `rolled-back` stays below $K = 2$. That count spans retained records for the node, not the current run alone, and K is not a universal limit on every explicit rollback route. At the threshold, or under an approval-required policy, the backstop records an escalation instead of writing, which bounds the implemented backstop decision rather than agent behavior in general.

F. Recipe Management and Rollback

Recipe management separates the definition of intended operation from the device state actually achieved. Versioned edits and known-good snapshots preserve candidate parameter sets and their provenance, and reverting a definition creates a new version with restored metadata without by itself restoring PLC values. Recipe application instead creates a run record and dispatches its parameters, while line start additionally

establishes the active line context; the two should not be conflated. During dispatch, deleted or disabled nodes, and nodes no longer attached to the recipe's line, produce explicit unsuccessful outcomes while the remaining parameters continue, so partial application stays distinguishable from complete. The ordinary active-window check resolves the recipe through the active run's identifier rather than reading every frozen run field at once; application passes its run identifier into controller writes, skips that window branch, and retains runtime range validation, so version history documents intent without establishing transactional consistency among definition, run metadata, and physical parameters.

Physical rollback selects a recorded baseline, anchor, batch-start, or known-good target and issues a new write the controller can reject, so a rollback verdict does not guarantee restoration. Across multiple nodes, recovery is compensating work: one restoration may succeed while another fails, and no distributed physical transaction erases intermediate process states, so operators must distinguish restored metadata, individual compensation, and verified process recovery.

G. Process-Parameter Mapping Layer

A parameter object projects a control node onto process meaning: it carries a key, a unit, a baseline interval, a product interval and the active recipe window, and it exposes no register, data type or driver-configuration field. A write through this surface is bounded by the intersection of node range, baseline interval, product interval and recipe window, so an agent can reason about a process quantity without seeing a transport address. The layer is an interface over the control path of Section IV-C rather than a second gate: it narrows the permitted interval and then enters the same admission checks.

H. Human-in-the-Loop Control

Human intervention attaches to concrete bindings and operations. *Manual* mode suspends execution for approval, while *auto* mode omits that wait but retains the applicable controller and runtime checks. The approval object presented to the operator names the requesting member, the node, the requested value, the effective interval after intersecting node range, recipe window, baseline interval and product interval, and the remaining time. An unbound member is rejected, and after approval the tool verifies that the binding still exists before the request enters the controller. Approval therefore permits a request to proceed; it reserves no device state and guarantees nothing about later checks. Tool approvals expire explicitly and resolve as denied when unanswered, so a silent expiry is indistinguishable from a rejection at the device and is not separately surfaced to the operator. Duplicate pending requests for the same agent and node are suppressed, and removing a binding can cancel outstanding requests. This timed approval object differs from an optimization record awaiting human action: a recorded escalation does not mean that recovery has executed, and the case of $K = 2$ being reached while no operator is present is covered by neither path.

Human decisions, agent proposals, and system recovery events provide complementary attribution, and that attribution

is useful only while intent, authorization, device outcome, and observed process effect stay distinguishable. Those distinctions are inspectable across the four operator surfaces of Fig. 4. Process-external engines also require deployment-level privilege and network restrictions, because tool bindings govern sanctioned tool calls, not arbitrary I/O by a compromised local process.

V. INTEGRATED EVALUATION

A. Evidence Design and the Reference Run

Four questions drive the evaluation. Can the platform commission heterogeneous protocol stacks and acquire from them? Does the governed write path reject requests that break engineering or recipe constraints without blocking legal ones? Can a fixed controller close the acquisition-actuation loop on a process model? Can a scripted team turn a production objective into governed actions that reach it? Source inspection, mock drivers, real protocol transports driving simulated plants, and historical LLM campaigns stay apart; nothing here comes from physical hardware or human operators, and no observed latency is an equivalence result.

Except where an archive is named, the results in this section come from one reference archived run, $B = 20260918043504\text{-bdo: seed 42, commit 7d4bfc0, hash 00e7dc827f4a2cea}$ over the harness modules, Node v24.19.0 on Windows x64. It holds 75 checks that all pass, with no warnings and no failures. The objective, control law, and probe definitions are not in the archive; they are fixed properties of the harness source named by that hash. The run therefore measures platform, protocol, governance, controller, and team mechanisms: the agent-tool loops use the deterministic `opcode` engine, and the LLM agent option is off. Three plants are exercised: a cast-film physics model, a strategy-based film line, and a 49-signal biaxial (BOPET) line.

B. Suite-Wide Results

Table I groups the suite's 17 phases into seven evidence layers. All 75 checks passed, and the harness's own weighted rubric scores the archive 100.0/100.

Three properties bound the table's meaning. The 24/24 and 9/9 figures describe configured collision cases, not boundary discrimination. The governance layers exercise specific branches of the write path, not every fault, storage condition, or escalation route. And three checks carry write sub-checks skipped because their nodes expose no writable setpoint, which the pass/warn/fail verdict does not surface.

C. Protocol Integration and Governed Writes

The platform provisions Modbus TCP, OPC UA, MQTT, and HTTP as writable lines and Modbus RTU over TCP as an acquisition-only satellite. Table II reports write latency and readback; the OPC UA readback entry is a single sample rather than a general claim of exactness.

The checks behind these numbers are the per-line integration checks `line-n-io` in phase P3 and `port-n-*` in phase P8.

TABLE I

INTEGRATED EVALUATION RESULTS IN ARCHIVE *B* (20260918043504-BDO). PHASES ARE THE SUITE'S OWN IDENTIFIERS. "WT" IS THE SCORECARD WEIGHT THE HARNESS ASSIGNS; "CHECKS" COUNTS PASS/TOTAL WITHIN THE LAYER. THE SEVEN LAYERS PARTITION ALL 17 PHASES AND ALL 75 CHECKS EXACTLY ONCE.

Evidence layer	Phases	Wt	Checks	Principal measured result
Plant model and provisioning	P0-P2	4	9/9	4 writable lines plus 1 satellite DAQ; $J^* = 89.894$ retrieved as the comparison denominator
Protocol I/O and governed write	P3	3	5/5	56 DAQ samples across 5/5 nodes; 24/24 probes rejected; 0/4 legal writes blocked; per-stack p50 mean 33.135 ms
Tool-level closed loop	P4	3	7/7	3 scripted setpoint iterations per line, each accepted and re-observed
Records, approval, recipe, parameter layer	P4b-P4f	10	12/12	Verdict separable from compensation; scripted approval in 16 ms; two recipe lifecycles; 4/4 parameter checks
AgentTeam missions	P4m, P10	6	16/16	Scripted mock teams: one mission reaches $204.704 \pm 0.75^\circ\text{C}$ in 1 write; the biaxial line reaches $25.68\ \mu\text{m}$ from $28.10\ \mu\text{m}$ in 3 writes on 2 protocols
Closed-loop evaluation and scenario reuse	P6, P8, P8b	9	22/22	$J/J^* = 0.963-0.972$ over three seeds; film-line 9/9 rejected, 0 false blocks; backstop restores baseline in 120.161 s
Multimodal acquisition and platform services	P7, P9	2	4/4	5 vector and 5 image frames; task dispatch COMPLETED; duplicate memory refresh stored once; 14 engines available
Total	17	37	75/75	0 warnings, 0 failures; hard gate green

TABLE II

PROTOCOL RESULTS IN *B*. WRITE LATENCY IS P50/P95 OVER SIX TIMED WRITES OF ONE VALUE TO ONE REGISTER PER STACK; WITH SIX SAMPLES THE P95 VALUE IS EFFECTIVELY THE OBSERVED MAXIMUM, SO THESE ARE LATENCY OBSERVATIONS AND NOT TAIL ESTIMATES. THE READBACK COLUMN IS A SINGLE POST-WRITE SAMPLE PER STACK, NOT SIX. REJECTION COUNTS SIX AUTHOR-DEFINED OUT-OF-CONSTRAINT PROBES PER WRITABLE STACK. RTU EXPOSES NO WRITABLE SETPOINT.

Stack	DAQ	p50/p95 (ms)	Readback	Reject
Modbus TCP	7	62.717/162.607	0.04 °C	6/6
OPC UA	11	18.711/21.899	exact	6/6
MQTT	11	18.837/19.140	N/A	6/6
HTTP	13	32.273/32.972	N/A	6/6
Modbus RTU over TCP	14	N/A	N/A	N/A

Each writable stack rejects four engineering-range and two recipe-window violations, giving 24/24 rejections, and none of the four designated legal writes is blocked. The probes are node-high plus 15 % of span, node-low minus 15 %, 10^6 , -50 , and the recipe window widened by 20 %, so the set shows rejection of gross violations and nothing about where the true limit lies. With six samples, and with the OPC UA readback contradicting "exact" on two earlier writes in the same archive, the readback column is an illustration rather than a characterisation. MQTT and HTTP publish or echo without returning a measured value, so their accepted writes are unverified rather than process confirmation.

The same run provisions the film-line scenario with three writable lines and two satellite nodes. Its nine out-of-constraint probes, three node-range probes per writable line, are all rejected and its three legal writes accepted, with no code change. That set exercises no recipe-window branch. Both scenarios come from the same simulator and the same five drivers, so this is configuration reuse within the existing vocabulary, not portability to an independent plant.

Reverting a recipe creates a new version rather than overwriting the old one; a known-good batch is retained; a recorded rollback verdict leaves the register untouched until a separate rollback call passes through the write path; and manual mode suspends an agent request until the scripted approval releases it, measured at 16 ms, which times the approval polling path rather than a human decision. The audit surface holds 11

parameter-change anchors and returns 200-entry audit and operations-log windows, so intent, approval, and execution are separately inspectable, though the journal is not a signed ledger and protocol I/O is not atomic with a concurrent recipe change.

A frozen four-arm ablation, archived separately (seed 42, three repetitions, commit d6c824d), shows what the write path contributes; its arms are full, no interlock, no read-back, and ungated. With the full path, all six attacks are rejected. With the batch-window interlock disabled, or with the path ungated, four are still rejected and the two recipe-window violations execute at the driver; disabling readback alone changes nothing. The engineering range is therefore structurally enforced and the batch-window check is the one that adds protection, while attribution records what happened without preventing it. The same archive carries boundary probes one tenth of a unit either side of the window edge: they are rejected where the window check is active, accepted where it is disabled, and no arm records a false block. The process-parameter layer passes 4/4 checks: its semantic surface exposes no register or driver-configuration field, writes are bounded by the intersection of node, parameter, product, and recipe limits, out-of-product and out-of-baseline values are rejected at their respective layers, the parameter reads back, and an unbound agent is denied. Agents thus receive a process-facing interface that does not disclose transport addresses; the layer is described in Section IV-G.

D. Fixed-Controller Closed-Loop Benchmark

The controller in this layer contains no LLM. It starts the cast-film twin at screw speed $N = 150\text{ rpm}$, line speed $v = 95\text{ m/min}$, and zone setpoint 200°C , and *maximizes* the objective

$$J = 55 J_{\text{thick}} + 25 J_{\text{quality}} + 8 J_{\text{energy}} + 7 J_{\text{throughput}}, \quad (3)$$

subject to melt temperature in $[195, 225]^\circ\text{C}$ and pressure at most 22 MPa. Each term is a normalized score in $[0, 1]$, so $J \in [0, 95]$: thickness quality saturates once $|h - 50| \leq 2\ \mu\text{m}$, and the other terms reward low defect rate, low screw speed, and high line speed. The controller updates $N \leftarrow N(50/h)$

under a 22-rpm step limit, holds line speed fixed, and corrects heating only when temperature leaves $[196, 224]$ °C. The objective, control law, and reference J^* come from the harness and the separately maintained simulator, not from the run. Two features of that scoring bound the ratio. Its throughput and energy terms are evaluated from the setpoints the controller issued rather than from measured values, because the twin returns no independent reading for them; and the control law inverts the model's own relation $h \propto N/v$, so the plateau is structural. The suite's gate is $J/J^* \geq 0.8$, so the reported value is not near a threshold.

The three seeds start at $J_0 = 70.115, 68.792$, and 70.248 . All six governed writes are accepted, none rejected, and the mean wall time is 36.193 s per seed. The archive reports an end-of-run score $J_{\text{end}} = 87.105, 87.027$, and 87.366 , defined as the mean of the last two evaluations, giving $J_{\text{end}}/J^* = 0.969, 0.968$, and 0.972 . The last measured evaluation is lower in every seed, at 86.921, 86.609, and 86.975, or 0.967, 0.963, and 0.968. The stopping rule requires two consecutive converged observations, and the first write already brought thickness inside the $50 \pm 2 \mu\text{m}$ plateau, so the second write follows from the rule and not from a further improvement: it raises defect rate from 0.38–0.53 % to 0.62–0.72 % and lowers the composite objective. Seed 44 shows the largest relative rise in defect rate. We therefore quote 0.963–0.972 across both conventions and do not describe the trajectory as monotone. Final thicknesses are 49.883, 49.383, and 49.317 μm .

Decomposing J at the final point shows the residual gap. The reference and the seeds both saturate the thickness term, so thickness contributes nothing to the difference; at seed 42's final point the seeds give up about 1.8 points on defect quality and 4.4 on screw-speed energy, but gain about 3.2 on line throughput, because the controller holds line speed at 95 m/min while the reference reaches $h = 49.18 \mu\text{m}$ at 40 m/min and 55 rpm. The gap therefore reflects the controller's restricted action set, not a failure to reach thickness. The reference is the best value on a finite grid whose resolution the archive does not record, so the ratios measure distance from that grid, and the two-write count belongs to this stopping rule, not to the controller.

E. Scripted Team Missions

Both missions are dispatched by *scripted* channel members: the harness creates the lead and the worker on its mock engine and the benchmark script issues every tool call itself, so no language model decides a setpoint. The missions exercise the path an agent would use, not agent performance. The first asks whether an objective filed on the channel task board can travel through team delegation, a governed write, and back to a closed task: bring a Modbus-TCP temperature node to 204.704 ± 0.75 °C in at most three governed writes after reading recent data. The lead dispatches the task, a bound worker performs a five-minute `daq_query`, and one `dcw_control` request raises the setpoint from 200 to 204.704. The readback returns 204.700, an optimization record opens and is judged keep, the journal attributes the write to the agent, and the parent task reaches COMPLETED. All six mission checks pass,

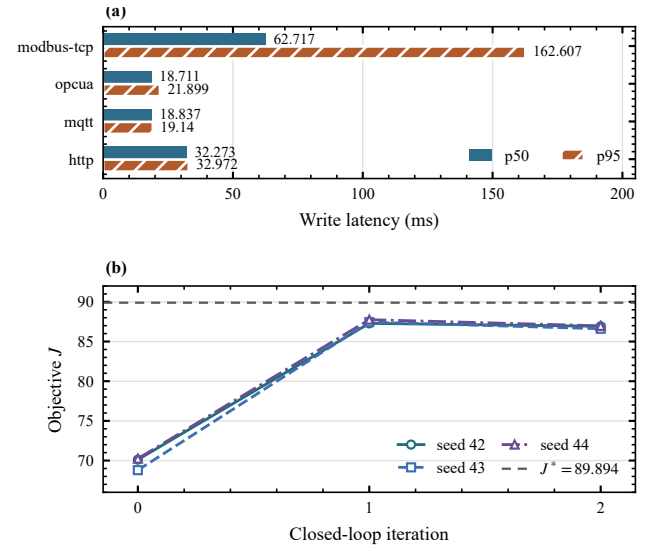


Fig. 5. Archive B. (a) Governed write latency p50 and p95 for the four writable stacks; Modbus TCP carries the longest tail. (b) Fixed-controller objective J for seeds 42–44, sampled at the start and after each of two governed writes, against the archived finite-grid reference $J^* = 89.894$. J is maximized. Each marker is one measured evaluation, so the third is the last one taken; it lies below the second because the second write raises defect rate. The archived J_{end} is a separate statistic, the mean of the last two evaluations, and is not plotted.

but the band is loose relative to the write, since a request for 204.0 would also have passed: the path closes, but the target was not hit precisely.

The second mission is the more demanding one. Fig. 6 maps it to the line, the team, the admission gate, the three recorded writes, and the response they produced. The biaxial-film preset is provisioned as one full line with nine devices, 30 writable setpoints, 19 configured acquisition nodes, and driver tests passing on all nine; all nine came from the benchmark's own probe-and-provision step, so this layer measures provisioning and governance, not commissioning a line that already existed. The scripted team files a thickness objective of $25.0 \pm 0.7 \mu\text{m}$, accepted as $|h - 25.0| \leq 0.7 \mu\text{m}$. The initial gauge reading, taken through the real MQTT path, is $28.10 \mu\text{m}$, outside the window. The worker then applies three governed actions:

cast-roll speed 32 $\rightarrow$ 33.8 m/min on the casting Modbus-TCP node brings thickness $28.10 \rightarrow 26.62 \mu\text{m}$ (opt-86313140); fast-roll speed 118 $\rightarrow$ 120.5 m/min on the MDO Modbus-TCP node brings it to $25.90 \mu\text{m}$ (opt-41929fe2); and exit rail width 3000 $\rightarrow$ 3031 mm on the TDO OPC UA node brings it to $25.68 \mu\text{m}$ (opt-8d2faadb). Each action is a separate governed write to a different line section, spanning two protocols and two engineering units. Every write cleared the gate shown in the figure (checks `biac-mission-multinode` and `biac-mission-attained`), was accepted by its driver, opened its own optimization record, and was followed by a gauge re-read after a settling wait. Only those parts are recorded: the archive stores no post-write readback value for these three writes, so we claim none. The verdicts are not an independent finding either, since the scripted caller issues keep unconditionally; the records show that judgment is

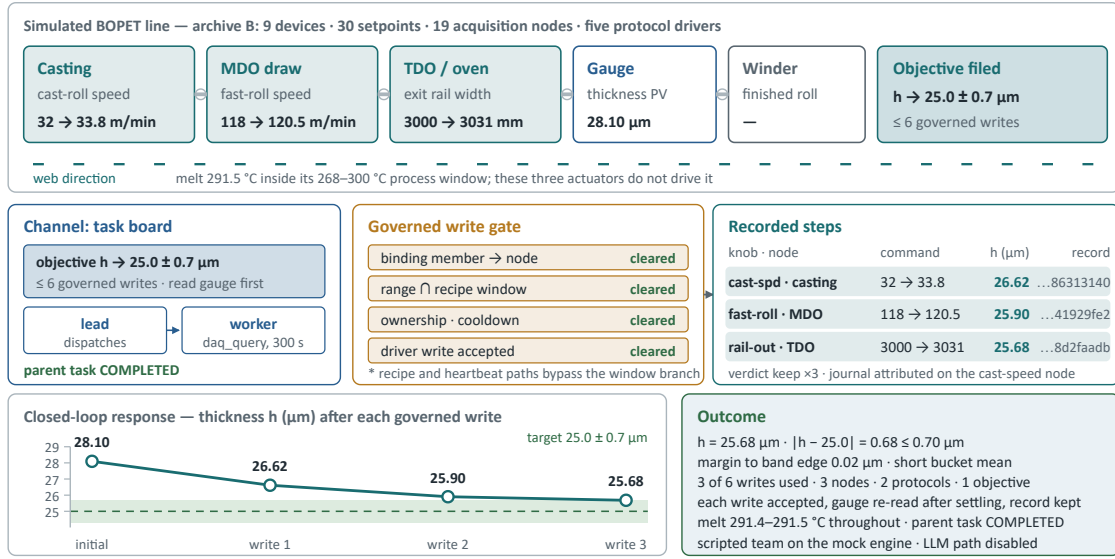


Fig. 6. Scripted AgentTeam mission in archive *B*, reconstructed from `agentteam-biax.log` and `run.json`. Top: the simulated BOPET line whose nine devices, 30 setpoints, and 19 acquisition nodes the benchmark provisions, with the three written actuators, the gauge that supplies thickness, and the objective filed against them. Middle: the channel that carries the objective, the four admission checks a governed agent write clears before a driver is invoked (Section IV-C sets out the recipe-application and heartbeat paths that bypass the window branch), and the three steps the run recorded. Bottom: measured thickness after each step against the $25.0 \pm 0.7 \mu\text{m}$ objective, and the outcome; the final point clears the band edge by $0.02 \mu\text{m}$. Team members are scripted mock agents, the setpoints come from a prespecified proportional rule, and the LLM agent is disabled, so the figure documents the governance and attribution path rather than an LLM’s optimization quality. The melt-temperature window is a simulator feasibility bound that these three actuators do not drive. The HTML source of this figure carries the same process as a step-by-step animation; the published frame is the complete static state.

stored separately from execution and compensation, not that a judgment was earned. Across the ten archived seed-42 runs of the reference command that report a final thickness, seven reached the band and three did not, and the run reported here is one of the seven. Two of the ten carry unrelated failing checks in other phases. The journal query for the cast-speed node returned the record with an agent source, which is the parameter-ledger surface of Section V-C; the separate mission check that asserts journal attribution is the one recorded with an empty evidence array in Section V-F.

The final thickness is $25.68 \mu\text{m}$, satisfying $|h - 25.0| = 0.68 \leq 0.7 \mu\text{m}$ by $0.02 \mu\text{m}$ on a short bucket mean, and the mission closes with 3 of its 6 allowed writes used. Melt temperature stays near 291.5 °C inside its $268\text{--}300 \text{ °C}$ process window, an observation rather than an enforcement result, since no disturbance tested that bound.

The mission is a scripted policy, not a search: each actuator carries a preset proportional gain applied to the relative thickness error, and the actuator is chosen by fixed rotation through the three knobs. The gains, 0.5, 0.35, and 0.3, were set deliberately low so that convergence takes three to four rounds and more than one node is involved; both the shrinking response (1.48, 0.72, and $0.22 \mu\text{m}$) and the three-write length follow from that configuration, not from emergence. What the mission does show is operational: the binding model resolves a different node, range, unit, and recipe window for each write, and the record structure keeps the three writes separate while attributing them to one objective. It does not establish autonomous strategy quality, cross-model generality, or physical-process performance.

F. Reproducibility, Recovery, and Threats to Validity

The backstop drill freezes a process variable outside its active window; the harness injects the breach rather than a drifting plant producing it. The system identifies an eligible open record, records a rollback judgment, and restores the recorded baseline in 120.161 s, a latency set by the backstop’s own wait-then-recheck cadence rather than by the plant. The drill shows one bounded recovery execution: it does not exercise escalation at $K = 2$, does not test detection of a slowly drifting excursion or of a gap between batches, and sets no worst-case recovery deadline. The same run stores five vector contour frames and five image frames, completes a lead-to-worker dispatch, stores a duplicate memory refresh once, and enumerates 14 registered harnesses, an inventory result that says nothing about control quality across those engines.

Three failures observed outside archive *B* bound the governance claim. During harness development, a burst of writes across several protocols drove the simulated plant into a collapsed state, with pump pressure, draw ratio, and thickness falling to their limits, while every platform-side write stayed readback-consistent. A readback therefore confirms that a driver delivered a value, not that the plant is in a sane state. A film-break guard that reapplies the recipe baseline is present in the mission harness but is not represented in archive *B*, which never entered that branch. Three other archives entered it, one of them earlier on the same day at the same recorded commit, and in each case the re-application failed to restore thickness, leaving 0.04 , -0.02 , and $-0.02 \mu\text{m}$. A fourth archive collapsed without entering the branch. The guard therefore aborts rather than recovers, and a negative thickness indicates that the plant model, not only the governance layer, bounds this result.

Third, the audit path fails open: bookkeeping errors are caught without reversing an executed write, so a physical change can occur without an attribution record, and the platform logs the error without blocking the write or retrying the record. Attribution is therefore a strong default, not a guarantee.

The command `(node bench/pipeline.mjs --profile integrated --seed 42 --cl-seeds 3)`, seed, commit, and a hash over the nine harness modules let another operator rerun the suite and compare judge-class outcomes against the same baseline. Five caveats keep this short of full reproducibility. The hash covers harness modules in the working tree, so it is sensitive to checkout line endings, and the plant simulator is a separately versioned dependency whose revision is not in the fingerprint, so the exact harness and plant behind these numbers cannot be reconstructed from the archive alone. The biaxial journal-attribution check is recorded as passing with an empty evidence array, so the run's own bookkeeping is not uniformly clean. And the checks were written against this implementation, with J^* drawn from the same simulator whose score function the controller optimises, so the gate is a development-time regression suite, not a benchmark with an external oracle. The last caveat concerns the mission layer. Its starting thickness is a simulator residue rather than a pinned initial condition, and later runs of the same command on the same host reached different outcomes: two archives sharing one harness hash and one seed, neither of them B , ended at $25.68\ \mu\text{m}$ and at $-0.02\ \mu\text{m}$; in the second, both mission checks are recorded as warnings, no check failed, and the hard gate stayed green. The suite is therefore not deterministic at the mission layer, and the 75/75 gate describes archive B rather than a property of the system. Verdicts and record states are deterministic within a run, protocol latencies are not, and the two should not be read as interchangeable. The evidence remains simulation-tier, and no physical device, TEP or SWaT replay, human-operator workload study, or external-framework comparison was run.

Historical real-LLM campaigns are exploratory evidence only. They use the same governed interface but different scoring inputs and temporal windows, the legacy script substituted missing observations, and the completion regex could match an instruction template, so their scores are not aggregated with the results above. Archive B uses the scripted path so that model variability does not enter the mechanism claims; where the platform's evaluation machinery proved weakest, in observation quality, we say so in Section VI.

VI. DISCUSSION AND LIMITATIONS

A. What the Integration Model Establishes

The reusable unit here is the deployed member-node capability-production-run association. It lets one task work in engineering quantities across drivers while execution and compensation stay attributable, and it lets the biaxial mission move three line sections under one objective without plant-specific routing code. The archive supports configuration reuse inside the existing driver and scenario vocabulary; it does not measure the effort to author a scenario or integrate a protocol the platform does not already speak.

Proposal, admission, and disposition stay distinguishable: a recorded keep is the agent's judgment, a matching readback is a driver-level observation, and sustained process convergence is a separate experimental outcome. The governed-write path concentrates range and recipe-window checks, ownership, cooldown, and readback, while the real-LLM campaigns exposed observation and scoring failures, where writes executed as designed but some judgments rested on stale or constant windows. Observation-quality gating is a framework-level extension this paper does not deliver, and until it exists the historical LLM scores cannot be compared with the deterministic ones.

B. Implementation and Evidence Boundaries

Cross-agent in-window oscillations, shared-node contention over longer horizons, and availability during storage failure remain unevaluated. Approval reduces permitted automation without establishing endpoint security, protecting against a privileged compromised harness, or validating approved engineering intent; least-privilege deployment and network separation remain separate responsibilities [35], [36]. Governed writes are defense-in-depth at the basic process control system layer, not a certified safety instrumented function: their safety argument presupposes independent regulatory limits and a protection layer in the IEC 61511 sense [37], and the platform implements no detection or degraded-mode response when such layers are absent, which is an integrator responsibility. Manual-binding approval workload grows with the frequency of agent proposals, and approval fatigue is a documented risk [27], [28]; policy granularity over roles, batch phases, and magnitudes is an extension point, not a present capability. None of these mechanisms is enforced outside the platform process, so they constrain only traffic that passes through it. At minimum, a deployment needs independent setpoint limits and interlocks in the controller itself.

Deployment in a real plant carries prerequisites this paper does not discharge. Placement across the IT/OT hierarchy and the assignment of IEC 62443 zones and conduits are integrator decisions [35]. A deployment design must also answer the platform's failure modes, including the approval channel, alarm paths, and open records left by a core crash, and must pin engine model versions where regulated industries require reproducible behavior. The single-host Windows capacity boundary is likewise untested.

VII. CONCLUSION

AGENTWORKSHOP integrates multi-agent coordination with heterogeneous industrial supervision through explicit node capabilities and shared production context. The architecture keeps coordination and supervision cooperating without becoming one atomic control function. Reference archive B provides three complementary results: four protocol fixtures reject all predefined out-of-constraint writes with no legal-write blocks; a fixed controller reaches $J/J^* = 0.963\text{--}0.972$ across three seeds; and a scripted team mission uses three governed writes across two protocols to bring biaxial-film thickness from 28.10 to $25.68\ \mu\text{m}$. These results demonstrate

the tested integration and governance paths only. Physical validation, stronger execution consistency, controlled agent comparisons, and observation-quality gating remain open.

DATA AVAILABILITY AND DISCLOSURE

The archive 20260918043504-bdo used in Section V is machine-readable: `run.json` holds every check with its evidence, `summary.json` the verdict and KPI set, `metrics.csv` the metric registry, and `agentteam-*.log` the mission traces. The figure scripts and the script that re-checks the quantitative claims against that archive are included. The source of Fig. 6 animates the same recorded process step by step, and the printed frame is its complete static state. Anonymized snapshots of the platform, benchmark card, and simulator accompany the submission; repository identifiers will be disclosed on acceptance. AI-based assistants supported manuscript editing, code-assisted evidence checking, and figure preparation under author direction, and the authors remain responsible for the claims and data.

REFERENCES

- [1] M. Raza, Z. Jahangir, M. B. Riaz, M. J. Saeed, and M. A. Sattar, "Industrial applications of large language models," *Scientific Reports*, vol. 15, 2025, art. no. 13755.
- [2] S. Colabianchi, F. Costantino, and N. Sabetta, "Assessment of a large language model based digital intelligent assistant in assembly manufacturing," *Computers in Industry*, vol. 162, 2024, art. no. 104129.
- [3] Z. Liu, R. Zeng, D. Wang, G. Peng, X. Liu, Q. Liu, P. Liu, W. Wang, and J. Wang, "Agents4PLC: Automating closed-loop PLC code generation and verification in industrial control systems using LLM-based agents," *IEEE Transactions on Software Engineering*, vol. 52, no. 5, pp. 1672–1687, 2026.
- [4] M. Fakih, R. Dharmaji, Y. Moghaddas, G. Quiros, O. Ogundare, and M. A. Al Faruque, "LLM4PLC: Harnessing large language models for verifiable programming of PLCs in industrial control systems," in *Proc. 46th Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2024, pp. 192–203.
- [5] J. Vyas, M. S. Gill, A. Markaj, F. Gehlhoff, and M. Mercangöz, "From detection to action: Using LLM agents for fault-tolerant control," *arXiv preprint arXiv:2606.28011*, 2026.
- [6] B. Hofmann, N. Piechulek, S. Kreitlein, J. Franke, and P. Bruendl, "Beyond touch-based human-machine interface: Control your machines in natural language by utilizing large language models and OPC UA," *arXiv preprint arXiv:2510.11300*, 2025.
- [7] P. Leitão, "Agent-based distributed manufacturing control: A critical survey," *Engineering Applications of Artificial Intelligence*, vol. 22, no. 7, pp. 973–989, 2009.
- [8] J. Zhang, H. Wang, Z. Jia, J. Dong, and L. Ren, "CoMA-IKG: LLM-driven multiagent framework for automated construction of industrial knowledge graph," *IEEE Transactions on Industrial Informatics*, vol. 22, no. 6, pp. 5553–5564, 2026.
- [9] P. Liu, L. Qian, X. Zhao, and B. Tao, "Joint knowledge graph and large language model for fault diagnosis and its application in aviation assembly," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 6, pp. 8160–8169, 2024.
- [10] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, "LLM experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins," in *Proc. IEEE ETFA*, 2024, arXiv preprint arXiv:2405.18092.
- [11] R. Wu, J. Ai, and T. Li, "InstructMPC: A human-llm-in-the-loop framework for context-aware control," *arXiv preprint arXiv:2504.05946*, 2025.
- [12] A. Biskupovic, A. Villalonga, F. Castaño, R. E. Haber, and F. Núñez, "Agent-based supervision for service-oriented industrial cyber-physical systems," *IEEE Transactions on Industrial Informatics*, vol. 21, no. 3, 2025.
- [13] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu, K. Chen, Q. Nie, and Y. Jia, "A large language model-based multi-agent manufacturing system for intelligent shopfloors," *Advanced Engineering Informatics*, vol. 69, 2026, art. no. 103888.
- [14] Q. Wu, G. Bansal, J. Zhang, T. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.
- [15] LangChain, Inc., "LangGraph," [Online]. Available: <https://github.com/langchain-ai/langgraph>, 2024.
- [16] CrewAI, Inc., "CrewAI," [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2024.
- [17] Anthropic, "Model context protocol," [Online]. Available: <https://modelcontextprotocol.io>, 2024.
- [18] Linux Foundation, "Agent2agent (A2A) protocol," [Online]. Available: <https://a2a-protocol.org>, 2025.
- [19] International Society of Automation, "Ansi/isa-95.00.01: Enterprise-control system integration," Standard, 2010.
- [20] —, "ANSI/ISA-88.01:2010—batch control part 1: Models and terminology (international equivalent IEC 61512-1)," Standard, 2010.
- [21] A. Y. C. Nee, S. K. Ong, G. Chrysosolouris, and D. Mourtzis, "Digital twin in industry: State-of-the-art," *IEEE Transactions on Industrial Informatics*, vol. 15, no. 4, pp. 2405–2418, 2019.
- [22] International Electrotechnical Commission, "Iec 63278 (series): Asset administration shell for industrial applications," International Standard, 2023.
- [23] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.
- [24] International Electrotechnical Commission, "IEC 62443-3-3: System security requirements and security levels," Standard, 2023.
- [25] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proc. AAAI Conference on Artificial Intelligence*, 2018.
- [26] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, 2001.
- [27] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, vol. 30, no. 3, pp. 286–297, 2000.
- [28] L. Bainbridge, "Ironies of automation," *Automatica*, vol. 19, no. 6, pp. 775–779, 1983.
- [29] X. Liu *et al.*, "AgentBench: Evaluating LLMs as agents," in *Proc. ICLR*, 2024, arXiv preprint arXiv:2308.03688.
- [30] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," *arXiv preprint arXiv:2406.12045*, 2024.
- [31] J. J. Downs and E. F. Vogel, "A plant-wide industrial process control problem," *Computers & Chemical Engineering*, vol. 17, no. 3, pp. 245–255, 1993.
- [32] J. Goh, S. Adepu, K. N. Junejo, and A. Mathur, "A dataset to support research in the design of secure water treatment systems," in *Proc. 11th Int. Conf. Critical Information Infrastructures Security (CRITIS)*, 2017, pp. 88–99.
- [33] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.
- [34] International Electrotechnical Commission, "IEC 62541: OPC unified architecture," International Standard, 2020.
- [35] —, "IEC 62443: Security of industrial automation and control systems," International Standard, 2018.
- [36] National Institute of Standards and Technology, "NIST SP 800-82 rev. 3: Guide to operational technology (OT) security," Special Publication, 2023.
- [37] International Electrotechnical Commission, "IEC 61511: Functional safety—safety instrumented systems for the process industry sector," International Standard, 2016.